Sign in

pytorch / **pytorch** Public

Tensors and Dynamic neural networks in Python with strong GPU acceleration

pytorch.org

View license

88.1k stars **23.7k** forks Branches Tags Activity

Star Notifications

bae6075

Go to file Code

apaszke Add basic THNN docs bae6075 · 9 years ago

doc	Add basic THNN docs	9 years ago
generic	Fix THNN header formatting	9 years ago
CMakeLists.txt	Change THNN library type to MODULE…	10 years ago
README.md	Add basic THNN docs	9 years ago
THNN.h	THNN: add missing OpenMP include	10 years ago
init.c	Add THNN conversion of Spatial*Conv…	9 years ago

README

THNN

THNN is a library that gathers nn's C implementations of neural network modules. It's entirely free of Lua dependency and therefore can be used in any application that has a C FFI. Please note that it only contains quite low level functions, and an object oriented C/C++ wrapper will be created soon as another library.

There is also a CUDA counterpart of THNN (CUTHNN) in the cunn repository.

Links

- API reference
- Style guidelines

Motivation

Torch's nn module provided many optimized C implementations of modules, but the source files contained Lua specific code and headers so they couldn't be easily compiled and included anywhere else.

THNN is based on the same code, but is written in pure C, so it can be easily included in other code. **Future C implementations should be committed to THNN.**

API

THNN is a purely functional library. It provides 2-3 functions for each module, that perform the most important operations:

- **updateOutput** - applies the module to an input
- **updateGradInput** - accepts gradient w.r.t. output and previous module input, and computes a gradient w.r.t. that input
- **accGradParameters** - *(optional, only modules with parameters)* accepts gradient w.r.t. output and previous module input, and computes gradient w.r.t. the parameters

For information on argument types please check the API reference.

This is all THNN library provides. An object oriented implementation similar to nn will be provided in a separate library. This one is just a set of CPU kernels.

Developer docs

- Style guidelines

This section will be expanded when FFI refactoring will be finished.

Releases 58

PyTorch 2.6.0 Release Latest
on Jan 29

+ 57 releases

Sign in

pytorch / pytorch Public

Code Issues 5k+ Pull requests 1.2k Actions Projects 12

Files bae6075

pytorch / doc /

api_reference.md

apaszke Add basic THNN docs bae6075 · 9 years ago

133 lines (114 loc) · 2.84 KB

Preview Code Blame Raw

This document only describes a THNN API. For a thorough review of all modules present here please refer to nn's docs.

Note on function names

Please remember, that because C doesn't support function overloading, functions taking different tensor types have different names. So e.g. for an Abs module, there are actually two updateOutput functions:

- `void THNN_FloatAbs_updateOutput(...)`
- `void THNN_DoubleAbs_updateOutput(...)`

In these docs such function will be referred to as `void THNN_Abs_updateOutput(...)`, and it's up to developer to add a type prefix. `real` is an alias for that type.

Module list

These are all modules implemented in THNN:

- Nonlinear functions
  - Abs
  - ELU
  - HardShrink
  - HardTanh
  - LeakyReLU
  - LogSigmoid
  - LogSoftMax
  - PReLU
  - RReLU
  - Sigmoid
  - SoftMax
  - SoftPlus
  - SoftShrink
  - Sqrt
  - Square
  - Tanh
  - Threshold
- Criterions
  - AbsCriterion
  - ClassNLLCriterion
  - DistKLDivCriterion
  - L1Cost
  - MSECriterion
  - MarginCriterion
  - MultiLabelMarginCriterion
  - MultiMarginCriterion
  - SmoothL1Criterion
- Modules
  - LookupTable
  - SparseLinear
- Spatial modules
  - SpatialAdaptiveMaxPooling
  - SpatialAdaptiveMaxPooling
  - SpatialAveragePooling
  - SpatialConvolutionMM
- Volumetric modules
  - VolumetricAveragePooling
  - VolumetricConvoluion

- VolumetricConvoluionMM
- VolumetricFullConvolution
- VolumetricMaxPooling
- VolumetricMaxUnpooling

Abs

```
void THNN_Abs_updateOutput(
          THNNState *state,
          THTensor *input,
          THTensor *output);
```

`state` - library's state
`input` - input tensor
`output` - **[OUT]** Abs output

```
void THNN_Abs_updateGradInput(
          THNNState *state,
          THTensor *input,
          THTensor *gradOutput,
          THTensor *gradInput);
```

`state` - library's state
`input` - input tensor
`gradOutput` - gradient w.r.t. output
`gradInput` - **[OUT]** gradient w.r.t. input

ELU

For reference see Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs).

```
void THNN_ELU_updateOutput(
          THNNState *state,
          THTensor *input,
          THTensor *output,
          real alpha);
```

`state` - library state
`input` - input tensor
`output` - **[OUT]** ELU output
`alpha` - an ELU parameter

```
void THNN_ELU_updateGradInput(
```

```
          THNNState *state,
          THTensor *input,
          THTensor *gradOutput,
          THTensor *gradInput,
          THTensor *output,
          real alpha);
```

`state` - library state
`input` - input tensor
`gradOutput` - gradient w.r.t. output
`gradInput` - **[OUT]** gradient w.r.t. input
`output` - module output for given input
`alpha` - an ELU parameter

Sign in

pytorch / pytorch Public

Code Issues 5k+ Pull requests 1.2k Actions Projects 12

Files bae6075

pytorch / doc /

style_guidelines.md

apaszke Add basic THNN docs bae6075 · 9 years ago

59 lines (48 loc) · 1.45 KB

- Preview
- Code
- Blame

Raw

Functions should return `void`.

All functions should accept arguments in the following order. `...` represent any module-specific parameters or buffers, disregarding whether they are used for writing or reading. Arguments in `...` below should be ordered like this:

```
[weight], [bias], [any buffers], [additional arguments], [optional argu
```

Modules

```
updateOutput: state, input, output, ...
updateGradInput: state, input, gradOutput, gradInput, ...
accGradParameters: state, input, gradOutput, [gradWeight], [gradBias],
```

e.g.

```
void THNN_(HardShrink_updateGradInput)(
          THNNState* state,
```

```
          THTensor *input,
          THTensor *gradOutput,
          THTensor *gradInput,
          real lambda)
```

Criterions

```
updateOutput: state, input, target, output, ...
updateGradInput: state, input, target, gradInput, ...
```

e.g.

```
void THNN_(ClassNLLCriterion_updateOutput)(
          THNNState* state,
          THTensor *input,
          THLongTensor *target,
          THTensor *output,
          THTensor *weights,
          THTensor *total_weight,
          bool sizeAverage)
```

Code style guide

```
void THNN_Linear_updateOutput(
          THTensor *input,
          THTensor *output,
          THTensor *weight,
          THTensor *bias);
//<- 10 ->
```

All arguments should start on a new line after function name, and they should be indented using 10 spaces.

Use 2 spaces for block indentation.